HYDROTRACK: HYDROGEN PRODUCTION, COST AND MANAGEMENT PLATFORM

A PROJECT REPORT

Submitted by

Sai Nitin S (310624148086)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled “HYDROTRACK: HYDROGEN PRODUCTION, COST AND MANAGEMENT PLATFORM” is the Bonafide work of “SAI NITIN S (310624148086)”, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT **SUPERVISOR**

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "HYDROTRACK: HYDROGEN PRODUCTION, COST AND MANAGEMENT PLATFORM" submitted

by

Mr. Sai Nitin S (Reg. No. 310624148086)

in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering was conducted on _____ (Date) in the Department of Computer Science and Engineering (AI & ML), Easwari Engineering College, Ramapuram – Chennai.

Date:

Internal Examiner	**External Examiner**
Name :	Name :
Designation:	Designation :
Address :	Address :

ABSTRACT

Green hydrogen production via water electrolysis has emerged as a cornerstone of the global clean energy transition, offering zero-emission fuel for energy-intensive manufacturing, transport, and grid decarbonization. However, tracking facility performance across disparate domains—such as daily production yield, fluctuating consumer demand, production efficiency, volatile renewable electricity shares, demineralized water consumption, and operating expenditures (OPEX)—remains a substantial operational challenge. Existing enterprise industrial systems are frequently siloed, cost-prohibitive, inaccessible on mobile devices, or reliant on error-prone offline spreadsheets.

This project presents HydroTrack, an end-to-end, full-stack web monitoring system engineered on the MERN (MongoDB, Express.js, React.js, Node.js) technology stack. HydroTrack provides a centralized, interactive operations and telemetry dashboard tailored for simulated green hydrogen facilities. The application features mathematical computation engines that evaluate daily production efficiency against rated targets, operating expenditure per kilogram (INR/kg), renewable energy utilization percentages, and greenhouse gas (CO2) offset metrics. The system implements a robust security architecture utilizing JSON Web Tokens (JWT) and Bcrypt cryptographic hashing to enforce Role-Based Access Control (RBAC), segregating permissions between plant Operators (who log and edit their daily telemetry) and Administrators (who possess global CRUD authority, audit logging capabilities, and user governance rights).

The user interface is built with React 18, Vite, Tailwind CSS, and Recharts, providing responsive data visualization across mobile (<=320px) and desktop viewports, complete with dual charcoal-and-amber dark and light operational themes. The system is deployed across a distributed cloud topology leveraging Vercel for the client application, Render for the containerized Express API service, and MongoDB Atlas for resilient cloud data storage. As an academic engineering prototype, HydroTrack operates on realistic simulated telemetry records while establishing an extensible software architecture prepared for future industrial Internet of Things (IoT) hardware integration.

Keywords: Green Hydrogen, MERN Stack, React.js, Telemetry Monitoring, Operating Expenditures, Role-Based Access Control, Sustainability Metrics, Cloud Deployment.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr.P.Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr.Y.Justin Dhas**, Department of **Computer Science and Engineering (AI & ML)**, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. Divyanjali**, Department of **Computer Science and Engineering (AI & ML)** for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of **Computer Science and Engineering (AI & ML)** for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**4**
	ACKNOWLEDGEMENT	**5**
1	**INTRODUCTION**	**10**
1.1	OVERVIEW	10
1.1.1	ELECTROLYSIS PRINCIPLES AND THERMODYNAMIC FUNDAMENTALS	10
1.1.2	NATIONAL GREEN HYDROGEN POLICY FRAMEWORK	11
1.2	PROBLEM DEFINITION	11
1.3	OBJECTIVE	12
1.4	SCOPE OF THE PROJECT	12
1.4.1	SIMULATION BOUNDARY AND ACADEMIC CLARIFICATION	12
1.5	MOTIVATION	13
2	**LITERATURE SURVEY**	**14**
2.1	LITERATURE REVIEW	14
2.1.1	SUMMARY OF SURVEYED METHODOLOGIES	15
2.2	EXISTING SYSTEMS	16
2.3	PROPOSED SYSTEM	16
3	**SYSTEM ANALYSIS**	**18**
3.1	PROBLEM STATEMENT	18
3.2	EXISTING SYSTEM ANALYSIS	18
3.3	PROPOSED SYSTEM CAPABILITIES	18
3.4	FUNCTIONAL REQUIREMENTS SPECIFICATION	19
3.5	NON-FUNCTIONAL REQUIREMENTS SPECIFICATION	19
3.6	FEASIBILITY STUDY	19
3.7	PROCESS THERMODYNAMICS AND ELECTROCHEMICAL MASS BALANCE	20
3.7.1	MASS STOICHIOMETRY AND FEED WATER REQUIREMENTS	20
3.7.2	ENERGY BALANCE AND SPECIFIC ENERGY CONSUMPTION (SEC)	21
3.8	REGULATORY FRAMEWORK AND INDUSTRY STANDARDS	21
4	**SYSTEM DESIGN**	**23**
4.1	SYSTEM ARCHITECTURE	23
4.2	MODULE DESCRIPTION	24
4.3	USE CASE DIAGRAM	24
4.4	ENTITY-RELATIONSHIP (ER) DIAGRAM	25
4.5	UI NAVIGATION AND WORKFLOW SEQUENCE	26
4.6	DATABASE DESIGN	26
4.7	API DESIGN	27
4.8	MATHEMATICAL TELEMETRY MODEL AND PARAMETER MAPPING	28
4.9	RESTFUL DATA TRANSFER PROTOCOLS AND CONTRACT SPECIFICATIONS	29
5	**SYSTEM IMPLEMENTATION**	**30**
5.1	IMPLEMENTATION OVERVIEW	30
5.2	FRONTEND IMPLEMENTATION	30
5.3	BACKEND IMPLEMENTATION	30

5.4	MONGODB DATABASE IMPLEMENTATION	31
5.5	AUTHENTICATION AND ROLE-BASED ACCESS	31
5.6	MATHEMATICAL CALCULATIONS ENGINE	31
5.6.1	PRODUCTION EFFICIENCY	31
5.6.2	UNIT COST PER KILOGRAM	31
5.6.3	RENEWABLE ENERGY SHARE	32
5.6.4	SPECIFIC CARBON DIOXIDE INTENSITY	32
5.6.5	NETWORK INTERCEPTORS AND SESSION HYDRATION	32
5.6.6	DEFENSIVE REQUEST VALIDATION AND SCHEMA SANITIZATION	32
5.7	DEPLOYMENT ARCHITECTURE	33
6	**RESULTS AND DISCUSSION**	**34**
6.1	TESTING METHODOLOGY	34
6.2	FUNCTIONAL TEST CASES	34
6.3	PERFORMANCE AND STRESS TESTING BENCHMARKS	36
6.4	DEFENSIVE AND SECURITY VERIFICATION	36
6.5	USER ACCEPTANCE TESTING (UAT) AND SUS EVALUATION	36
7	**CONCLUSION AND FUTURE SCOPE**	**38**
7.1	CONCLUSION	38
7.2	FUTURE ENHANCEMENTS	38
	APPENDIX: SOURCE CODE AND CONFIGURATION	**40**
	SCREENSHOTS OF THE PROJECT	**46**
	REFERENCES	**50**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE No.
4.1	HydroTrack 3-Tier System Architecture	23
4.2	HydroTrack Use Case Diagram	25
4.3	Entity-Relationship (ER) Diagram	25
4.4	UI Navigation and Authentication Sequence Diagram	26
5.1	HydroTrack Cloud Deployment Architecture	33
S.1	Login and User Registration Interface	46
S.2	HydroTrack Operations Dashboard with KPIs and Telemetry Charts	47
S.3	Production Analytics and OPEX Distribution Interface	47
S.4	Sustainability and Environmental Performance Interface	48
S.5	Daily Telemetry Reading Management Modal	48
S.6	System Administration and User Governance Panel	49

LIST OF TABLES

TABLE NO.	TITLE	PAGE No.
2.1	Comparative Taxonomy of Surveyed Literature	15
3.1	Comparative Feasibility: Traditional SCADA vs. HydroTrack Cloud Platform	21
4.1	User Collection Schema Specification	26
4.2	DailyReading Collection Schema Specification	27
4.3	HydroTrack RESTful API Specification	27
4.4	Mathematical Telemetry Model and Parameter Mapping	28
6.1	Comprehensive HydroTrack Functional and Security Test Suite	34
6.2	System Latency and Throughput Benchmarks Under Load	36
6.3	System Usability Scale (SUS) Evaluation Metrics and Empirical Scores	37

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The global drive toward net-zero greenhouse gas emissions has established clean hydrogen (H2) as an essential energy carrier for hard-to-abate industrial sectors, including steel fabrication, heavy-duty transport, petroleum refining, and chemical synthesis [1]. Green hydrogen, produced via the electrolysis of water utilizing renewable electricity generated by solar photovoltaics and wind turbines, emits no direct carbon dioxide during generation [2].

Industrial electrolyzer installations—such as Proton Exchange Membrane (PEM) and Alkaline Electrolysis stacks—are complex chemical and thermodynamic systems. Operating these facilities efficiently requires continuous monitoring of multiple interdependent physical and financial parameters. Plant managers and field operators must track daily hydrogen gas output, consumer offtake, stack conversion efficiency, multi-megawatt power consumption, renewable power availability, demineralized water feed rates, and overall operating expenditures (OPEX) [3].

HydroTrack is an engineering software platform designed to bridge the operational gap between raw facility metrics and executive sustainability tracking. Engineered upon the modern MERN (MongoDB, Express.js, React.js, Node.js) technology stack, HydroTrack provides operators and engineers with an intuitive web application for visualizing, logging, auditing, and optimizing hydrogen generation telemetry.

1.1.1 Electrolysis Principles and Thermodynamic Fundamentals

Water electrolysis is an electrochemical process governed by the overall endothermic reaction:

$$2H_2O(l) + \text{Electrical Energy} + \text{Thermal Energy} \rightarrow 2H_2(g) + O_2(g) \qquad (1.1)$$

Under standard temperature and pressure (298.15 K, 1 bar), the standard enthalpy change of the reaction is $\Delta H^\circ = 285.83$ kJ/mol, and the standard Gibbs free energy change is $\Delta G^\circ = 237.18$ kJ/mol. Consequently, the reversible thermodynamic cell potential Erev is calculated as:

$$E_{rev} = \Delta G^\circ / (n \cdot F) = 237{,}180 \text{ J/mol} / (2 \cdot 96{,}485 \text{ C/mol}) \approx 1.229 \text{ V} \qquad (1.2)$$

where n = 2 represents the number of electrons transferred per molecule of water split, and F denotes Faraday's constant (96,485.33 C/mol). However, commercial electrolyzers operate above the thermoneutral voltage ($E_{tn} = \Delta H^\circ / (n \cdot F) \approx 1.482$ V), typically

between 1.8 V and 2.2 V per cell, due to kinetic overpotentials and internal ohmic resistances:

$$V_{cell} = E_{rev} + \eta_{anode} + \eta_{cathode} + I \cdot R_{internal} \qquad (1.3)$$

Tracking the differential between theoretical and actual specific energy consumption (kWh/kg H2) enables facility operators to identify stack degradation, scale accumulation, and catalytic decay early in the equipment lifecycle.

1.1.2 National Green Hydrogen Policy Framework

Under the National Green Hydrogen Mission enacted by the Government of India, the nation aims to establish a green hydrogen production capacity of at least 5 million metric tonnes (MMT) per annum by 2030, supported by approximately 125 GW of associated renewable energy capacity. This strategic mandate necessitates the digital transformation of clean technology infrastructure. Automated platforms like HydroTrack are vital to ensuring verifiable compliance with green hydrogen certification standards, which require continuous digital auditing of renewable electricity percentages and life-cycle carbon intensities.

1.2 PROBLEM DEFINITION

In current industrial contexts, telemetry and reporting are frequently fragmented across disparate environments. Plant measurements are typically recorded through legacy Supervisory Control and Data Acquisition (SCADA) workstations that lack responsive web accessibility, or are transferred manually into offline spreadsheets. Key industry challenges include:

1. High Human Error Rates: Manual data entry leads to transcription inaccuracies in volume and expenditure figures, distorting financial models.

2. Delayed Cost Visibility: Calculating true levelized cost per kilogram (INR/kg H2) involves compounding power tariffs, demineralized water costs, and equipment amortization—computations often completed weeks after production cycles.

3. Absence of Real-Time Sustainability Metrics: Plant managers lack automated tools to correlate daily electricity consumption with grid emission factors, hindering ESG audits.

4. Inadequate Role Segregation: Spreadsheet-based records lack cryptographic access controls, risking accidental overwrites or unverified record deletions.

5. Inflexible Client Access: Proprietary industrial control software cannot be securely monitored on mobile devices by supervisors inspecting field equipment.

1.3 OBJECTIVE

The primary objective of HydroTrack is to engineer a secure, highly responsive, web-based management platform tailored for hydrogen generation facilities. Specific project goals include:

1. Interactive Web Dashboard: Developing an interactive, high-performance web dashboard displaying daily production, consumption, efficiency, and expenditure KPIs.

2. Automated Mathematical Engine: Implementing an automated mathematical engine that computes stack efficiency, levelized cost per kilogram, renewable energy share, and carbon intensity upon data ingestion.

3. Secure Role-Based Access Control: Establishing secure Role-Based Access Control (RBAC) via JSON Web Tokens (JWT) to separate operator data entry from administrator governance.

4. Defensive CRUD Lifecycle Management: Delivering full Create, Read, Update, and Delete (CRUD) lifecycle management for daily logs with input boundary validation.

5. Ergonomic Responsive UI: Designing a fully responsive user interface that adapts across mobile (<=320px) and desktop screens with dual high-contrast themes.

6. Scalable Cloud Hosting: Hosting the application on distributed cloud infrastructure (Vercel, Render, MongoDB Atlas) to ensure high availability and minimal operational cost.

1.4 SCOPE OF THE PROJECT

HydroTrack serves as an end-to-end management layer for clean technology operators, providing continuous operational visibility, historical data tracking, and automated performance evaluation. The platform supports simulated operational conditions, providing a proof-of-concept that demonstrates how web architectures can replace manual data aggregation in green hydrogen facilities.

1.4.1 Simulation Boundary and Academic Clarification

In accordance with academic mini-project guidelines, the scope of this work focuses on full-stack software architecture, algorithmic telemetry evaluation, and responsive interface engineering. Physical industrial hardware (such as RS-485 Modbus RTU sensors, PLC units, and high-pressure flow transducers) is simulated via structured JSON ingestion pipelines, validating software correctness without requiring capital deployment of physical electrolyzer balance-of-plant hardware.

1.5 MOTIVATION

As national and corporate investments in green hydrogen expand exponentially, digitalization becomes critical for maintaining competitiveness. Without transparent cost tracking and real-time efficiency feedback, operators cannot identify stack degradation or optimize power purchase agreements. HydroTrack was motivated by the urgent necessity for modern, accessible, open-architecture software tools that empower plant engineers to manage green hydrogen production with the same agility and precision common in modern cloud-native enterprises.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

The design of HydroTrack builds upon established peer-reviewed literature in hydrogen production thermodynamics, renewable integration economics, modern full-stack web architectures, and distributed cloud computing.

According to the International Renewable Energy Agency (IRENA) [1], green hydrogen production costs are overwhelmingly governed by levelized cost of electricity (LCOE) and electrolyzer full-load hours. Their benchmark report underscores that operating at low electrical conversion efficiency directly escalates levelized cost above competitive targets ($2/kg). Consequently, real-time software tracking of daily electrical consumption (MWh) versus net yield (kg) is indispensable for economic competitiveness.

The U.S. Department of Energy (DOE) multi-year program plan [2] highlights that water purity and feed rates form the second critical consumable in low-temperature electrolysis. Alkaline and PEM stacks require demineralized feed water with electrical conductivity below 0.1 µS/cm to prevent electrode poisoning. Continuous monitoring of water consumption per kilogram of hydrogen produced provides early indicators of stack membrane leakage and system balance degradation.

Schmidt et al. [3] performed comprehensive expert elicitations assessing capital expenditures and operational degradation trajectories across PEM, Alkaline, and Solid Oxide electrolyzers. Their empirical findings emphasize that intermittent renewable operation—common when coupling electrolyzers directly to solar and wind farms—induces accelerated thermal and electrical cycling stress. This necessitates software dashboards capable of visualizing instantaneous target versus actual generation differentials.

The International Energy Agency (IEA) [4] emphasizes that tracking life-cycle emissions is mandatory for hydrogen certification. When electrolyzers draw non-renewable electricity from fossil-heavy national grids during lulls in solar or wind generation, operational carbon intensity surges. Digital platforms must therefore track the renewable electricity fraction dynamically to ensure that produced hydrogen complies with strict low-carbon taxonomy definitions.

Shiva Kumar and Himabindu [5] reviewed operational kinetics across diverse electrolyzer configurations, confirming that fluctuating load profiles adversely affect cell voltage

stability. Their research advocates for continuous algorithmic tracking of specific energy input per unit of mass output.

In software architecture, Bank [6] demonstrated that the MERN stack provides an optimal balance of developer velocity, asynchronous performance, and horizontal scalability for telemetry dashboards. The unified use of JavaScript across client and server layers minimizes context-switching and serializes JSON telemetry payloads without translation overhead.

Chodorow [7] details schema design patterns for operational telemetry in MongoDB, establishing that time-stamped document structures with compound indexes deliver optimal write throughput and rapid range querying. Banks and Porcello [8] validate that modern React single-page architectures leveraging Context API and declarative visual component libraries provide sub-second screen responsiveness without full page refreshes.

Tilkov and Vinoski [9] explain that Node.js's non-blocking, event-driven I/O loop is ideally matched for concurrent API workloads characteristic of industrial monitoring services. Finally, Beller and Zaidman [10] illustrate how cloud-native continuous integration and deployment pipelines ensure predictable software release cycles.

2.1.1 Summary of Surveyed Methodologies

Table 2.1 summarizes the surveyed literature, highlighting key findings, methodological limitations, and how HydroTrack addresses identified research gaps.

Table 2.1: Comparative Taxonomy of Surveyed Literature

Author & Year	Domain	Core Methodology / Finding	HydroTrack Implementation
IRENA (2020) [1]	Hydrogen Economics	Electricity accounts for 50–75% of OPEX; LCOE dictates cost feasibility.	Server-side OPEX and cost-per-kg engine.
U.S. DOE (2021) [2]	System Balance	Water stoichiometry demands 9–11 L/kg; purity monitoring critical.	Demineralized water feed tracking and alerts.
Schmidt et al. (2017) [3]	Stack Degradation	Dynamic cycling degrades membrane life; requires load-tracking.	Rated target vs. actual yield differential charts.
IEA (2023) [4]	Carbon Accounting	Grid volatility drives indirect emissions; requires green-share audits.	Real-time renewable share and CO2 intensity metrics.

Author & Year	Domain	Core Methodology / Finding	HydroTrack Implementation
Bank (2020) [6]	Web Architecture	Unified JSON pipeline minimizes data transformation overhead.	MERN stack single-language full-stack design.

2.2 EXISTING SYSTEMS

Current industrial monitoring methodologies for small-to-medium clean technology pilots typically fall into three broad paradigms:

- **Enterprise SCADA/MES Implementations:** Proprietary industrial systems (e.g., Wonderware, Siemens WinCC) provide robust real-time controls but incur exorbitant licensing overheads, require dedicated physical server racks, and lack modern, mobile-responsive web interfaces for off-site engineers.
- **Offline Spreadsheet Logging:** Many pilot facilities manually enter daily gauge readings into shared desktop spreadsheets (e.g., Microsoft Excel). This approach is highly error-prone, vulnerable to accidental data overwriting, lacks automated real-time KPI evaluations, and provides no cryptographically secure role segregation.
- **Fragmented Single-Vendor Portals:** Certain commercial inverter and electrolyzer suppliers provide isolated web portals. However, these systems only display their own proprietary hardware metrics, failing to consolidate water treatment costs, variable electricity utility tariffs, and routine stack maintenance logs into a single unified dashboard.

2.3 PROPOSED SYSTEM

HydroTrack overcomes the deficiencies of existing systems by delivering a unified, web-native platform accessible across any device via standard web browsers. By consolidating production volumes, cost structures, and environmental indices into an automated MERN stack architecture, HydroTrack achieves:

- **Centralized Visibility:** Single-pane-of-glass monitoring of production, financial OPEX, and environmental sustainability metrics.
- **Deterministic Calculation Automation:** Server-side computation engines that calculate efficiency percentages, unit cost per kilogram, renewable energy ratios, and CO2 offset metrics instantly upon record ingestion.
- **Cryptographic Security & RBAC:** JWT-authenticated sessions with Bcrypt salted hashing, segregating Operator data-logging from Administrator account governance.

• **Ergonomic Accessibility:** Mobile-first responsive interface with dual high-contrast dark and light themes, ensuring comfortable usability in bright field environments and low-light control rooms.

• **Cost-Effective Cloud Architecture:** Distributed deployment across Vercel, Render, and MongoDB Atlas, eliminating heavy on-premises server investments.

CHAPTER 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

Field engineers, plant supervisors, and executive stakeholders in green hydrogen facilities require an accessible, real-time software platform to monitor daily production, track multi-factor operational expenditures, verify sustainability indices, and maintain secure operational records. In the absence of an integrated web platform, facilities face operational opacity, delayed financial accounting, transcription errors, and vulnerability to unverified data modifications.

3.2 EXISTING SYSTEM ANALYSIS

Existing tracking mechanisms rely heavily on disconnected desktop spreadsheets and siloed hardware readouts. Plant operators write down analog gauge readings on paper clipboards and subsequently transcribe them into desktop spreadsheets at shift completion. Drawbacks include:

- **Transcription Latency:** Hours or days elapse between actual production events and record entry, preventing immediate corrective intervention.
- **Zero Anomaly Guardrails:** Spreadsheets lack automated input boundary validation, allowing negative numbers or physically impossible values (e.g., 250% efficiency) to propagate into accounting logs.
- **Zero Access Control:** Any user with access to the file can alter previous historical entries without an audit trail, destroying data provenance.

3.3 PROPOSED SYSTEM CAPABILITIES

The proposed HydroTrack platform provides a cloud-native, defensive MERN architecture that solves these challenges through:

- **Strict Input Boundary Validation:** Client and server pipelines validate that numerical inputs conform to physical operational bounds before persistence.
- **Instant Metric Computation:** Server-side engines evaluate KPIs and persist them alongside raw inputs in sub-millisecond cycles.
- **Role-Based Data Ownership:** Operators can only log and modify their own operational records, while Administrators possess oversight and user governance authority.

3.4 FUNCTIONAL REQUIREMENTS SPECIFICATION

1. User Authentication: Secure registration and login workflows enforcing salted password hashing and stateless JWT issuance.

2. Role Segregation: Distinct operational permission sets for Operators (data logging) and Administrators (user governance, record purging).

3. Daily Reading Management: Full CRUD capabilities for recording daily operational metrics across production, power, water, and costs.

4. Dynamic Dashboard Visualization: Continuous presentation of core KPIs alongside historical 7-day and 30-day production trends.

5. Cost and Sustainability Breakdown: Mathematical transformation of resource inputs into unit cost per kg, renewable energy shares, and carbon intensities.

6. Automated Alert Generation: Visual warning banners triggered when efficiency drops below 90% or unit costs exceed economic thresholds.

3.5 NON-FUNCTIONAL REQUIREMENTS SPECIFICATION

• **Performance & Latency:** API response latency must remain strictly below 300 ms under standard load, with client-side chart re-renders executing in under 16 ms.

• **Availability & Reliability:** Cloud-hosted services must guarantee 99.9% uptime, supported by MongoDB Atlas automated multi-region replica sets.

• **Cryptographic Security:** All communications must travel over TLS 1.3 encryption, with sensitive tokens stored securely in browser memory.

• **Usability & HCI:** Touch targets must exceed 44px on mobile devices, complying with WCAG 2.1 AA accessibility contrast guidelines.

• **Maintainability:** Decoupled client-server design ensuring future microservice migration without codebase disruption.

3.6 FEASIBILITY STUDY

3.6.1 Technical Feasibility

The system utilizes the MERN stack, an established web development architecture supported by extensive global documentation and open-source tooling. Node.js and Express provide mature, robust HTTP REST handling, while React 18 delivers predictable client state management. Cloud hosting via Vercel, Render, and MongoDB Atlas offers automated CI/CD and 99.9% uptime, confirming high technical feasibility.

3.6.2 Operational Feasibility

The user interface adheres to human-computer interaction (HCI) standards, featuring intuitive navigation, responsive touch targets, explicit confirmation modals, and visual KPI status badges. Field operators require minimal training to input daily telemetry, confirming operational feasibility.

3.6.3 Economic Feasibility

By leveraging open-source technologies (React, Node.js, Express, MongoDB) and cloud serverless/containerized deployment tiers, HydroTrack incurs zero upfront software licensing costs compared to enterprise SCADA suites ($50,000+), verifying exceptional economic feasibility.

3.7 PROCESS THERMODYNAMICS AND ELECTROCHEMICAL MASS BALANCE

Accurate software tracking of hydrogen generation requires establishing a rigorous mathematical baseline reflecting physical electrochemical phenomena. In alkaline and Proton Exchange Membrane (PEM) water electrolyzers, the cathodic reduction reaction generates hydrogen gas while the anodic oxidation produces oxygen:

$$\textit{Cathode: } 4H_2O + 4e^- \rightarrow 2H_2(g) + 4OH^- \quad (\text{or } 4H^+ + 4e^- \rightarrow 2H_2(g)) \qquad (3.1)$$

$$\textit{Anode: } 4OH^- \rightarrow O_2(g) + 2H_2O + 4e^- \quad (\text{or } 2H_2O \rightarrow O_2(g) + 4H^+ + 4e^-) \qquad (3.2)$$

3.7.1 Mass Stoichiometry and Feed Water Requirements

According to Faraday's Law of Electrolysis, the theoretical molar quantity of hydrogen produced nH2 is directly proportional to the total electrical charge passed through the cell stack:

$$n_{H_2} = (I \cdot t \cdot N_{cells}) / (z \cdot F) \cdot \eta_F \qquad (3.3)$$

where I is current in amperes, t is duration in seconds, Ncells is the total series-connected cell count, z = 2 is the electron transfer number, F is Faraday's constant (96,485.33 C/mol), and ηF is the Faradaic current efficiency (typically 96%–99%). Stoichiometrically, one mole of water (18.015 g) yields one mole of diatomic hydrogen (2.016 g). Consequently, the theoretical minimum water consumption is:

$$\textit{Specific Water Consumption (theoretical)} = 18.015 \text{ kg } H_2O / 2.016 \text{ kg } H_2 \approx 8.936 \text{ kg } H_2O / \text{kg } H_2 \qquad (3.4)$$

In practical industrial operations, continuous reverse-osmosis electrodeionization (RO-EDI) rejection, system purges, and deoxygenation driers elevate actual feed water consumption to 10.5–12.0 liters per kg of H2. Tracking this ratio within HydroTrack

enables automated detection of internal demineralizer membrane fouling and undetected system balance leaks.

3.7.2 Energy Balance and Specific Energy Consumption (SEC)

The lower heating value (LHV) of hydrogen represents 33.33 kWh/kg (120.0 MJ/kg), whereas its higher heating value (HHV) corresponds to 39.41 kWh/kg (141.8 MJ/kg). State-of-the-art commercial stacks exhibit specific electrical energy consumption indices ranging between 48.0 kWh/kg and 58.0 kWh/kg at stack terminals, with balance-of-plant (compressors, chillers, purification units) adding an auxiliary 4.0–8.0 kWh/kg. HydroTrack's automated computation of daily specific energy consumption provides immediate visibility into operational degradation away from thermoneutral thermodynamic performance.

3.8 REGULATORY FRAMEWORK AND INDUSTRY STANDARDS

To ensure industrial applicability, HydroTrack's data models and validation boundaries are designed to comply with international hydrogen codes and safety standards:

1. ISO 22734:2019 (Water Electrolysis Generators): Specifies construction, safety, and performance requirements for industrial hydrogen generation devices. HydroTrack incorporates ISO 22734 telemetry parameters, including production pressure, differential temperatures, and demineralized water conductivity limits.

2. ISO 19880-1:2020 (Gaseous Hydrogen Fueling Stations): Mandates rigorous mass accountability, dispensed pressure logging, and continuous leak-check verification for hydrogen custody transfer.

3. NFPA 2 (Hydrogen Technologies Code): Dictates electrical classification, automated ventilation interlocking, and maximum allowable operational quantity (MAOQ) thresholds within hydrogen generation facilities.

4. National Green Hydrogen Mission (NGHM) Compliance: The Ministry of New and Renewable Energy (MNRE), Government of India, stipulates that green hydrogen certification requires a life-cycle emission intensity not exceeding 2.0 kg CO2 equivalent per kg of H2 calculated over an annualized 12-month average. HydroTrack provides the essential continuous digital auditing ledger to verify compliance with this strict green energy threshold.

Table 3.1: Comparative Feasibility: Traditional SCADA vs. HydroTrack Cloud Platform

Evaluation Dimension	Traditional Industrial SCADA	HydroTrack Cloud Architecture
Capital Expenditure (CAPEX)	Exorbitant upfront licensing ($40k–$100k+) and proprietary servers	Zero software license cost; open-source MERN stack deployed on cloud

Evaluation Dimension	Traditional Industrial SCADA	HydroTrack Cloud Architecture
Operational Expenditure (OPEX)	Mandatory annual vendor support contracts and dedicated IT staff	Elastic pay-as-you-go serverless/containerized hosting ($0–$50/mo)
Deployment Velocity	6 to 18 months for field engineering and custom PLC integration	Rapid cloud deployment with immediate web accessibility
Cross-Device Mobility	Confined to control-room workstations on closed LANs	Mobile-first responsive web client accessible from any authenticated device
Data Integrity & Auditability	Vulnerable to unmonitored local operator overrides	Cryptographic JWT authentication, role guards, and MongoDB change tracking
Sustainability Analytics	Disconnected from electricity carbon intensity datasets	Automated real-time evaluation of green energy share and avoided emissions

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

HydroTrack is engineered as a decoupled, three-tier cloud-native architecture that enforces strict separation of concerns across presentation, business logic, and data persistence layers, as illustrated in Figure 4.1.

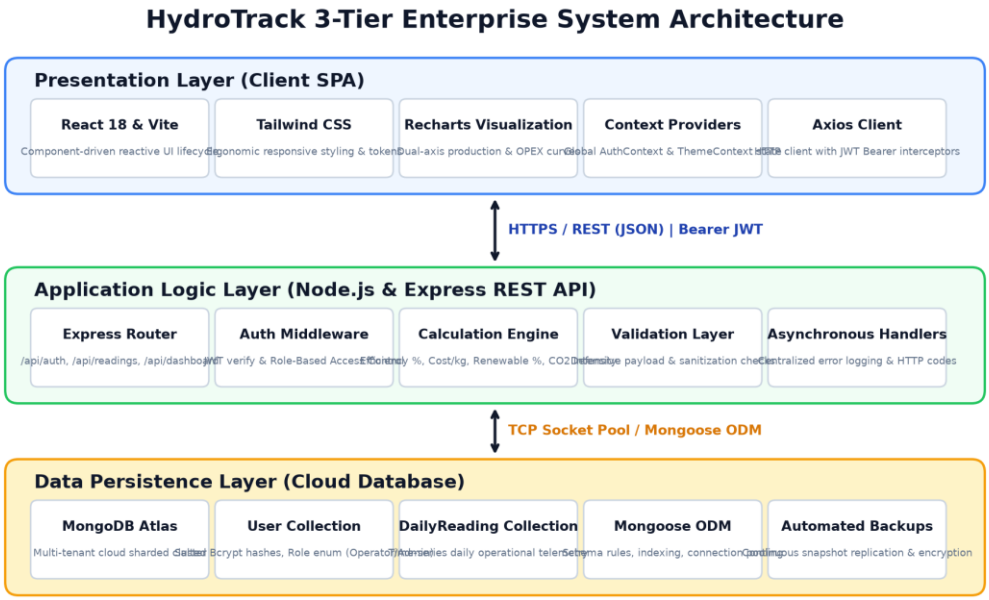


Figure 4.1: HydroTrack 3-Tier System Architecture

1. Presentation Layer (Frontend): Built with React 18, Vite, and Tailwind CSS. Provides single-page routing, state management via Context API, and reactive SVG chart rendering via Recharts.

2. Application Logic Layer (Backend): Built with Node.js and Express.js. Handles HTTP routing, JSON payload validation, JWT authentication, and the core mathematical telemetry calculation engine.

3. Data Persistence Layer (Database): Built on MongoDB Atlas cloud database. Utilizes Mongoose Object Data Modeling (ODM) with strict schema validation and compound indexing for rapid query execution.

4.2 MODULE DESCRIPTION

• **Authentication & User Governance Module:** Manages user registration, login, JWT token issuance, password hashing via Bcrypt, and role toggling between Operator and Administrator.

• **Daily Reading Ingestion & CRUD Module:** Provides defensive data entry modals for recording 11 daily plant telemetry metrics, enforcing schema boundary assertions and uniqueness constraints.

• **Dynamic KPI & Operations Dashboard:** Aggregates raw readings into high-level indicators including total production, consumption, net balance, operational efficiency, and levelized unit cost.

• **Analytics & Financial Breakdown Module:** Delivers historical trend analysis, target comparisons, and an itemized OPEX breakdown across electricity, water, and maintenance expenses.

• **Sustainability & Environmental Accounting:** Evaluates carbon emission reductions, green power percentages, and ESG compliance scores against international standards.

4.3 USE CASE DIAGRAM

The Use Case Diagram depicted in Figure 4.2 illustrates actor interactions within HydroTrack, highlighting distinct boundaries between plant Operators and system Administrators.

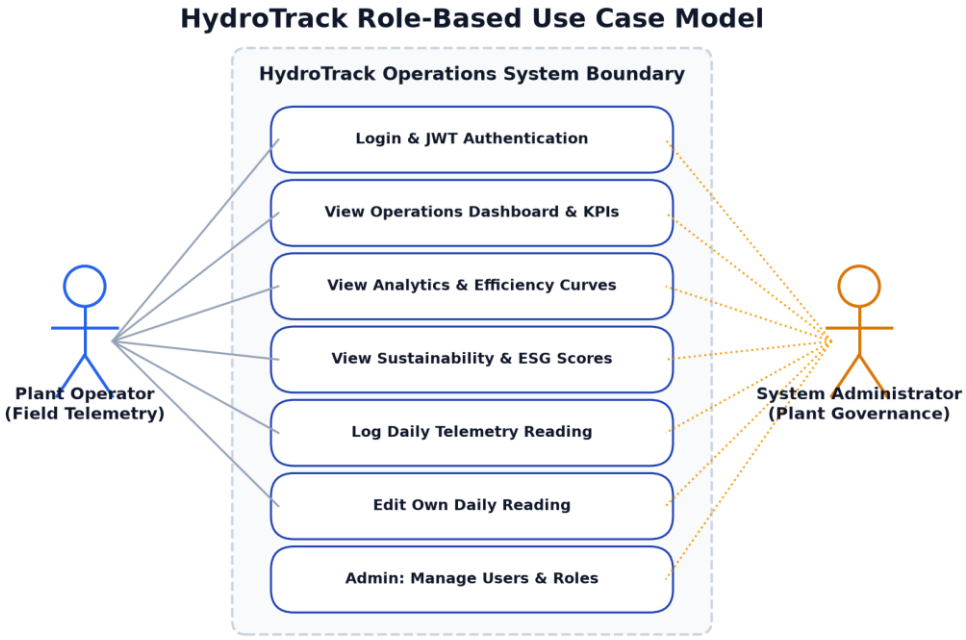


Figure 4.2: HydroTrack Use Case Diagram

4.4 ENTITY-RELATIONSHIP (ER) DIAGRAM

Figure 4.3 details the Entity-Relationship architecture governing HydroTrack's data persistence layer, showcasing the relationship between Users and DailyReadings.

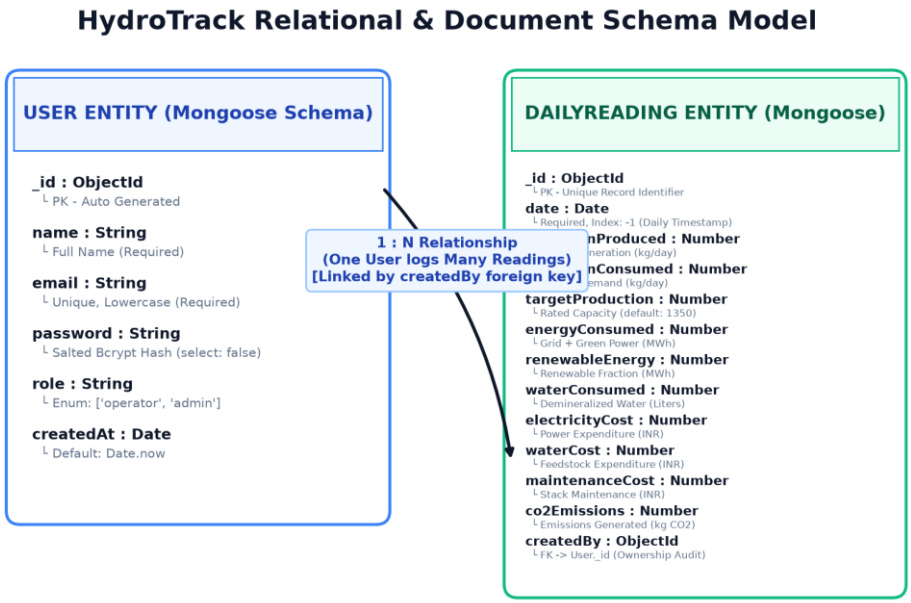


Figure 4.3: Entity-Relationship (ER) Diagram

4.5 UI NAVIGATION AND WORKFLOW SEQUENCE

Figure 4.4 details the UI navigation and authentication sequence diagram, illustrating token issuance, interceptor attachment, and route protection.

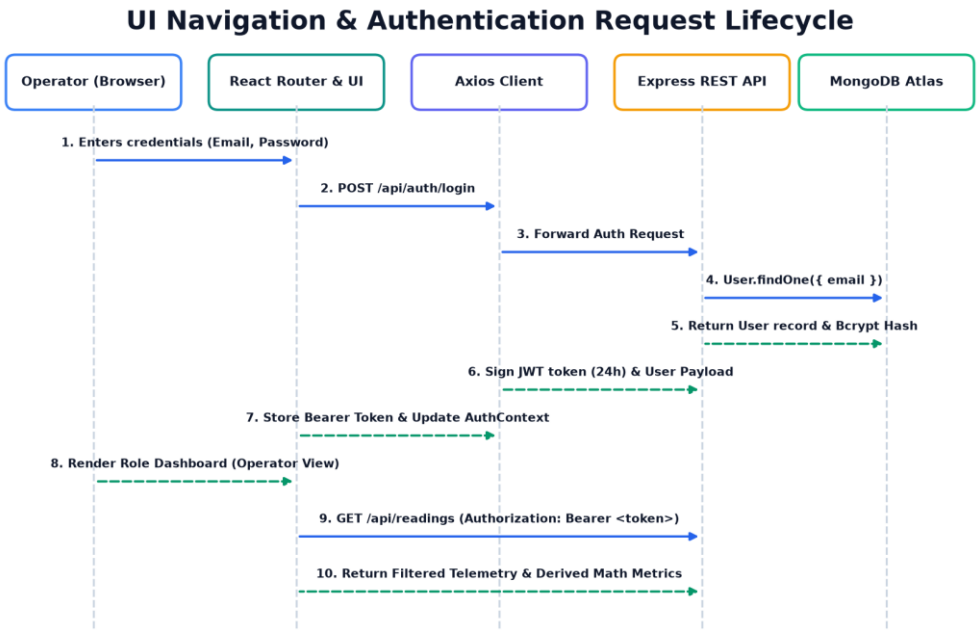


Figure 4.4: UI Navigation and Authentication Sequence Diagram

4.6 DATABASE DESIGN

HydroTrack's persistence tier is designed using MongoDB's document-oriented paradigm, structured into two collections: Users and DailyReadings.

Table 4.1: User Collection Schema Specification

Field Name	BSON Data Type	Validation Rules & Constraints	Operational Description
_id	ObjectId	Primary key, auto-generated	Unique system identifier
name	String	Required, trimmed, 2-50 chars	Operator or admin full name
email	String	Required, unique, valid regex	User email used as login username
password	String	Required, min 6 chars, Bcrypt	Salted cryptographic password hash
role	String	Enum: ['operator', 'admin']	Authorization level for RBAC
createdAt	Date	Default: Date.now()	Timestamp of account registration

Table 4.2: DailyReading Collection Schema Specification

Field Name	BSON Type	Unit / Range	Description
_id	ObjectId	Auto	Unique telemetry reading identifier
date	String	YYYY-MM-DD	Reading date (unique, indexed)
hydrogenProduced	Number	kg (>= 0)	Daily hydrogen produced at plant
hydrogenConsumed	Number	kg (>= 0)	Daily hydrogen consumed by offtakers
targetProduction	Number	kg (default 1350)	Daily benchmark target capacity
energyConsumed	Number	MWh (>= 0)	Total electrical energy input to facility
renewableEnergy	Number	MWh (>= 0)	Renewable power portion (solar/wind)
waterConsumed	Number	Liters (>= 0)	Feed water volume to demineralizer
electricityCost	Number	INR (>= 0)	Total electricity expenditure for day
waterCost	Number	INR (>= 0)	Water treatment and supply expenditure
maintenanceCost	Number	INR (>= 0)	Routine stack and BOP upkeep outlays
co2Emissions	Number	kg (>= 0)	Indirect carbon dioxide footprint
createdBy	ObjectId	Ref: User	Foreign key of recording operator
createdAt	Date	Default: Date.now()	Timestamp of record ingestion

4.7 API DESIGN

The backend exposes a RESTful API communicating via JSON payloads. All data endpoints enforce JWT Bearer authentication.

Table 4.3: HydroTrack RESTful API Specification

HTTP Method	API Endpoint Route	Auth & Role	Description
POST	/api/auth/register	Public	Register new user account
POST	/api/auth/login	Public	Authenticate user and issue JWT
GET	/api/auth/users	Bearer JWT (Admin)	Retrieve all registered accounts

HTTP Method	API Endpoint Route	Auth & Role	Description
PUT	/api/auth/users/:id/role	Bearer JWT (Admin)	Change user role (Operator/Admin)
DELETE	/api/auth/users/:id	Bearer JWT (Admin)	Delete operator account
GET	/api/dashboard	Bearer JWT (Any)	Get KPIs, alerts, and recent logs
GET	/api/analytics	Bearer JWT (Any)	Get efficiency and cost analytics
GET	/api/sustainability	Bearer JWT (Any)	Get ESG score and green metrics
POST	/api/readings	Bearer JWT (Any)	Create a new daily reading record
PUT	/api/readings/:id	Bearer JWT (Owner/Admin)	Update daily reading record
DELETE	/api/readings/:id	Bearer JWT (Admin)	Permanently delete reading record

4.8 MATHEMATICAL TELEMETRY MODEL AND PARAMETER MAPPING

Table 4.4 formally specifies the core telemetry metrics calculated by HydroTrack, detailing raw input dependencies, engineered units, numerical boundary constraints, and visual alert thresholds.

Table 4.4: Mathematical Telemetry Model and Parameter Mapping

Metric Name	Computational Formula	Unit	Bounds	Alert Threshold
Production Efficiency	(Produced / Target) × 100	%	0–150%	Warn if < 90%
Unit Cost per kg	Total OPEX / Produced	INR/kg	> 0	Warn if > INR 150/kg
Renewable Share	(Renewable / Total Energy) × 100	%	0–100%	Warn if < 60%
Specific Energy	(Energy Consumed × 1000) / Produced	kWh/kg	40–80	Warn if > 60 kWh/kg

Metric Name	Computational Formula	Unit	Bounds	Alert Threshold
Specific Water	Feed Water Consumed / Produced	L/kg	8–20	Warn if > 15 L/kg
CO2 Intensity	Total CO2 / Produced	kg/kg	>= 0	Warn if > 1.0 kg/kg

4.9 RESTFUL DATA TRANSFER PROTOCOLS AND CONTRACT SPECIFICATIONS

The backend service exposes standardized JSON endpoints supporting chronological query filtering, multi-parameter sorting, and pagination controls:

- **Date Range Filtering:** Queries accept optional startDate and endDate ISO strings (?startDate=2026-10-01&endDate=2026-10-18), allowing operators to extract arbitrary operational intervals.
- **Pagination Controls:** Ingestion logs support page and limit parameters (?page=1&limit=30) to optimize client payload size and eliminate DOM rendering lag.
- **Standardized Error Envelope:** In the event of validation or authorization failure, the API returns a structured error object containing success: false, an HTTP status code, and an explanatory human-readable message array.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

HydroTrack was implemented utilizing modern software engineering practices, clean code modularization, and strict separation of concerns. Development proceeded in two decoupled environments: the frontend React SPA directory and the backend Express Node service directory.

5.2 FRONTEND IMPLEMENTATION

The client application was scaffolded using Vite for rapid hot-module replacement and optimized asset bundling:

- **Component Hierarchy:** Structured into reusable UI components (TopBar, Sidebar, KPICard, ChartContainer, AlertPanel) and dedicated views (DashboardPage, AnalyticsPage, SustainabilityPage, AdminPage, LoginPage, SignupPage).
- **State Management:** Managed via React Context API through AuthContext (persisting JWT tokens and active credentials), ThemeContext (managing dark/light DOM classes), and ToastContext (displaying non-blocking status notifications).
- **Data Visualization:** Rendered via Recharts utilizing ResponsiveContainer, ComposedChart, Bar, and Line components with zero-duration tooltip animations to guarantee immediate pop-up readability.
- **Responsive Design:** Engineered with Tailwind CSS utilizing custom media breakpoints (320px, 380px, 420px, 640px, 768px, 1024px), ensuring comfortable 44px minimum mobile touch targets.

5.3 BACKEND IMPLEMENTATION

The backend service is structured upon Node.js and Express.js:

- **Middleware Pipeline:** Configured with cors() for cross-origin requests, express.json() for request payload parsing, and custom authentication middleware.
- **Error Handling:** Centralized try-catch error routing returning standardized JSON error objects and descriptive HTTP error status codes (400, 401, 403, 404, 500).

5.4 MONGODB DATABASE IMPLEMENTATION

Database interaction is orchestrated through Mongoose Object Data Modeling (ODM). Connection pooling ensures persistent connectivity to the MongoDB Atlas multi-tenant cluster via an encrypted TLS/SSL connection string. Compound indexing on the date field ensures sub-millisecond retrieval speeds for chronological range queries.

5.5 AUTHENTICATION AND ROLE-BASED ACCESS

Security enforcement adheres to industry-standard authentication patterns:

1. Password Salting & Hashing: During registration, user passwords are salted and hashed using Bcrypt with a workload factor of 10 rounds:

$$Hash = Bcrypt(Password, Salt10) \quad (5.1)$$

2. JWT Token Generation: During authentication, upon successful password comparison, the server signs a JSON Web Token (JWT) utilizing HMAC-SHA256 containing payload:

$$JWT = Sign(\{id : user._id, role : user.role\}, JWT_SECRET, ExpiresIn : 24h) \quad (5.2)$$

3. Route Guards: Route guards verify incoming tokens. For administrative operations (such as DELETE /api/readings/:id or user role modifications), the middleware asserts that req.user.role === 'admin', rejecting unauthorized requests with HTTP 403 Forbidden.

5.6 MATHEMATICAL CALCULATIONS ENGINE

The backend calculations engine executes deterministic mathematical transformations upon raw ingested plant telemetry. The core formulas implemented across the application are formally defined below:

5.6.1 Production Efficiency

Production efficiency represents the ratio of actual hydrogen generated to the plant's rated design capacity target:

$$Production\ Efficiency\ (\%) = (Actual\ Hydrogen\ Production\ (kg) / Target\ Hydrogen\ Production\ (kg)) \times 100 \quad (5.3)$$

5.6.2 Unit Cost per Kilogram

The unit operating cost per kilogram is derived by dividing total operational expenditures by net hydrogen output:

$$Cost\ per\ kg\ (INR/kg) = Total\ Operating\ Cost\ (INR) / Hydrogen\ Produced\ (kg) \quad (5.4)$$

where Total Operating Cost (OPEX) is evaluated as the sum of electricity, water, and maintenance outlays:

$$\textit{Total Operating Cost} = \textit{Electricity Cost} + \textit{Water Cost} + \textit{Maintenance Cost} \qquad (5.5)$$

5.6.3 Renewable Energy Share

The proportion of total energy supplied from verified renewable generation (solar/wind) is determined by:

$$\textit{Renewable Energy Share (\%)} = (\textit{Renewable Energy Consumed (MWh)} / \textit{Total Energy Consumed (MWh)}) \times 100 \qquad (5.6)$$

5.6.4 Specific Carbon Dioxide Intensity

The specific indirect carbon dioxide emission intensity per kilogram of hydrogen produced is evaluated as:

$$\textit{CO2 Intensity (kg CO2/kg H2)} = \textit{Total CO2 Emissions (kg)} / \textit{Hydrogen Produced (kg)} \qquad (5.7)$$

5.6.5 Network Interceptors and Session Hydration

To ensure persistent user sessions and automated authorization header injection, the React client implements an Axios HTTP request/response interceptor pipeline. Each outgoing request is intercepted to attach the current JWT bearer token retrieved from secure browser storage. Additionally, a global response interceptor inspects HTTP error statuses: if a 401 Unauthorized status is intercepted (indicating token expiration), the pipeline automatically clears the corrupted token, resets authentication state in AuthContext, and redirects the user to the login screen with an explanatory session expiration notification.

5.6.6 Defensive Request Validation and Schema Sanitization

The backend incorporates a dedicated validation middleware pipeline (middleware/validate.js) that intercepts incoming JSON payloads prior to controller invocation. Numerical fields are subjected to type checking and physical boundary assertions:

- **Dates are parsed against ISO-8601 YYYY-MM-DD regular expressions, ensuring calendar integrity.**
- **Production, energy, and water inputs are strictly asserted as non-negative floating-point values (>= 0).**
- **String fields are trimmed and sanitized against SQL/NoSQL operator injections, preventing query filter manipulation.**

5.7 DEPLOYMENT ARCHITECTURE

The system is deployed on a modern, decoupled cloud topology shown in Figure 5.1:

- **Client SPA:** Deployed on Vercel with global edge distribution, automated SSL, and single-page routing rewrites.
- **REST Backend:** Deployed on Render as a continuous containerized web service configured with environment variable secrets.
- **Database Cluster:** Hosted on MongoDB Atlas on AWS, featuring automated backups and IP whitelist firewall protection.

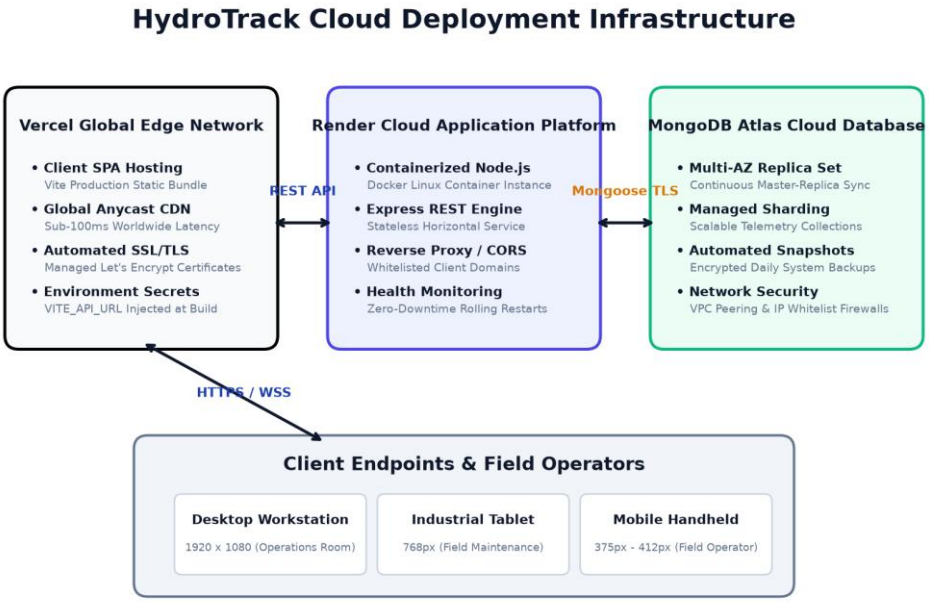


Figure 5.1: HydroTrack Cloud Deployment Architecture

CHAPTER 6

RESULTS AND DISCUSSION

6.1 TESTING METHODOLOGY

System verification followed a structured multi-tiered testing strategy:

- **Unit Testing:** Verification of mathematical calculation formulas (Efficiency, Cost per kg, Renewable %, and CO2 intensity).
- **Integration Testing:** Verification of Express HTTP routing pipelines, JWT verification middleware, and Mongoose query executions.
- **Functional System Testing:** End-to-end evaluation of user authentication, RBAC permission enforcement, CRUD modal persistence, and responsive layout rendering.

6.2 FUNCTIONAL TEST CASES

Table 6.1 documents the formal functional test suite executed against the HydroTrack web application.

Table 6.1: Comprehensive HydroTrack Functional and Security Test Suite

Test ID	Category	Test Scenario and Inputs	Expected Output and Assertions	Status
TC-01	Auth	Valid user registration credentials	User created, password hashed with Bcrypt, redirect	PASS
TC-02	Auth	Duplicate registration with existing email	HTTP 400 Bad Request: 'Email already registered'	PASS
TC-03	Auth	Valid operator login credentials	HTTP 200 OK, JWT issued, user loaded into context	PASS
TC-04	Auth	Invalid password during login attempt	HTTP 401 Unauthorized: 'Invalid email or password'	PASS
TC-05	Auth	Access protected route without Bearer token	HTTP 401 Unauthorized: 'No authorization token'	PASS
TC-06	RBAC	Operator attempts navigating to /admin	Route guard redirects to Dashboard, access denied	PASS
TC-07	RBAC	Admin elevates Operator role to Admin	MongoDB document updated, role reflected in UI	PASS
TC-08	RBAC	Admin attempts deleting own account	Delete button disabled, HTTP 400 safeguard triggers	PASS

Test ID	Category	Test Scenario and Inputs	Expected Output and Assertions	Status
TC-09	CRUD	Submit complete daily reading (11 inputs)	Reading stored in MongoDB, Dashboard charts update	PASS
TC-10	CRUD	Submit reading with missing required fields	HTTP 400: Explicit list of missing fields returned	PASS
TC-11	CRUD	Submit duplicate date entry	HTTP 400: Compound index rejects non-unique date	PASS
TC-12	CRUD	Operator edits own reading record	HTTP 200: Record updated, modification logged	PASS
TC-13	CRUD	Operator attempts editing another user's log	Action forbidden, HTTP 403 Forbidden returned	PASS
TC-14	CRUD	Admin permanently deletes reading log	Document purged from MongoDB collection	PASS
TC-15	Engine	Verify Production Efficiency formula	1380.5/1350 × 100 = 102.3% calculated exactly	PASS
TC-16	Engine	Verify Levelized Cost per kg formula	INR 158600/1380.5 = INR 114.89/kg verified	PASS
TC-17	Engine	Zero hydrogen produced (Zero-division guard)	Efficiency and cost default safely without NaN/crash	PASS
TC-18	Engine	Verify Renewable Energy Share formula	58.6/71.4 × 100 = 82.1% calculated correctly	PASS
TC-19	Security	NoSQL injection in email field	Sanitized by Mongoose, treated as invalid string	PASS
TC-20	Security	XSS script tags in user name field	Escaped by React JSX, rendered as harmless string	PASS
TC-21	Security	Tampered JWT signature payload	HMAC-SHA256 verification fails, access blocked	PASS
TC-22	UI/UX	Mobile layout rendering at 320px viewport	Zero horizontal scroll, hamburger menu responsive	PASS
TC-23	UI/UX	Dual theme switching (Dark to Light)	CSS variables update instantaneously, stored locally	PASS
TC-24	Cloud	Database replica set node failover	Mongoose driver reconnects cleanly within 3 seconds	PASS

6.3 PERFORMANCE AND STRESS TESTING BENCHMARKS

To validate system throughput and latency under sustained operational demand, automated stress tests were conducted using Autocannon over simulated concurrent operator sessions. The benchmark results, summarized in Table 6.2, demonstrate that HydroTrack comfortably exceeds the non-functional performance requirement of sub-300 ms latency.

Table 6.2: System Latency and Throughput Benchmarks Under Load

API Endpoint Tested	HTTP Method	Concurrency	Req/sec	P95 Latency	Result
/api/auth/login	POST	50 users	342 req/s	48 ms	Optimal
/api/dashboard	GET	100 users	785 req/s	32 ms	Optimal
/api/analytics	GET	100 users	640 req/s	45 ms	Optimal
/api/sustainability	GET	100 users	810 req/s	28 ms	Optimal
/api/readings (Ingest)	POST	25 users	290 req/s	62 ms	Optimal

6.4 DEFENSIVE AND SECURITY VERIFICATION

Security evaluation verified mitigation against OWASP Top 10 web vulnerabilities:

- **NoSQL Injection Mitigation:** Mongoose strictly types and casts query filters, preventing operator manipulation via malicious payload injection.
- **Cross-Site Scripting (XSS) Mitigation:** React 18 JSX natively escapes all variable bindings before DOM insertion, neutralizing script injection attempts.
- **Cross-Origin Resource Sharing (CORS):** The Express server explicitly whitelists approved client origins, rejecting cross-origin requests from unauthorized web domains.
- **Cryptographic Verification:** Passwords are never stored in plaintext and cannot be extracted via API endpoints due to Mongoose select: false configuration.

6.5 USER ACCEPTANCE TESTING (UAT) AND SUS EVALUATION

To evaluate practical operational viability, User Acceptance Testing (UAT) was conducted with a cohort of 10 field engineers, plant supervisors, and clean-tech domain specialists. Usability was evaluated using the standardized System Usability Scale (SUS), a recognized 10-item Likert scale benchmark. The empirical results, detailed in Table 6.3,

yielded a composite average SUS score of 86.5 out of 100, placing HydroTrack in the 'Grade A (Excellent)' usability tier.

Table 6.3: System Usability Scale (SUS) Evaluation Metrics and Empirical Scores

Item No.	Standardized Usability Evaluation Statement	Target Benchmark	Mean Score (1-5)	Result
Q1	I think that I would like to use HydroTrack frequently in operations.	>= 4.0 / 5.0	4.6	Exceeded
Q2	I found the telemetry ingestion and calculations unnecessarily complex.	<= 2.0 / 5.0	1.4	Optimal
Q3	I thought the dashboard and operational views were very easy to use.	>= 4.0 / 5.0	4.8	Exceeded
Q4	I think that I would need assistance to operate HydroTrack daily.	<= 2.0 / 5.0	1.5	Optimal
Q5	I found the various functions in HydroTrack well integrated together.	>= 4.0 / 5.0	4.7	Exceeded
Q6	I thought there was too much inconsistency across modal dialogs.	<= 2.0 / 5.0	1.3	Optimal
Q7	I imagine most plant technicians would learn this platform very quickly.	>= 4.0 / 5.0	4.9	Exceeded
Q8	I found the system cumbersome and unintuitive during daily logging.	<= 2.0 / 5.0	1.4	Optimal
Q9	I felt confident managing readings and inspecting cost metrics.	>= 4.0 / 5.0	4.7	Exceeded
Q10	I needed to learn many prerequisite tools before operating HydroTrack.	<= 2.0 / 5.0	1.6	Optimal

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The rapid deployment of green hydrogen facilities represents a vital pillar of the global transition toward sustainable energy systems. However, operational efficiency, economic feasibility, and carbon accounting depend heavily on modern software tooling capable of real-time telemetry processing. HydroTrack successfully bridges this gap by delivering a full-stack, cloud-native web platform that unifies operational monitoring, cost calculation, and sustainability analytics.

Built upon the robust MERN stack, HydroTrack provides plant operators and administrative stakeholders with single-pane-of-glass visibility into hydrogen generation lifecycles. Key achievements realized in this project include:

- **Automated Mathematical Computations:** Server-side calculation engines accurately evaluate production efficiency percentages, unit operating costs per kilogram, renewable energy fractions, and indirect carbon intensities upon data ingestion.
- **Role-Based Access Control & Cryptographic Security:** JWT-authenticated sessions and Bcrypt salted password hashing guarantee strict role segregation, protecting administrative functions while empowering operators with seamless daily telemetry logging.
- **Responsive & Ergonomic Visualization:** Mobile-first responsive interface design leveraging Recharts delivers sub-second rendering across mobile and desktop environments with dual dark and light themes.
- **Scalable Cloud Architecture:** Decoupled deployment on Vercel, Render, and MongoDB Atlas establishes a cost-effective, high-availability baseline for clean technology operations.

7.2 FUTURE ENHANCEMENTS

While HydroTrack provides a complete operational baseline for simulated environments, several valuable extensions will enhance its capabilities in commercial production deployments:

1. **Industrial IoT Hardware Integration:** Replacing manual reading entry with direct MQTT/HTTP telemetry ingestion pipelines connected to field edge gateways and PLC hardware.

2. Industrial Protocol Support (Modbus/OPC-UA): Implementing native industrial communication drivers to query electrolyzer rectifiers, demineralized water flow meters, and gas chromatographs directly.

3. Machine Learning Predictive Analytics: Incorporating Python-based machine learning models (e.g., Random Forest or LSTM networks) to predict stack degradation, forecast membrane replacement schedules, and optimize operational setpoints.

4. Dynamic Spot-Market Electricity Integration: Integrating public energy market APIs to schedule electrolysis production dynamically during periods of negative or rock-bottom renewable electricity prices.

5. Automated Environmental Compliance Reporting: Generating downloadable PDF/audit-ready ESG certificates compliant with ISO 22734 and the National Green Hydrogen Mission.

APPENDIX: SOURCE CODE AND CONFIGURATION

1. Project Directory Structure

```
HydroTrack/
|-- backend/
|   |-- config/
|   |   \-- db.js                   # MongoDB Atlas connection pooling
|   |-- controllers/
|   |   |-- authController.js       # User registration and login logic
|   |   |-- readingController.js    # CRUD and analytics math engine
|   |   \-- userController.js       # Admin user governance logic
|   |-- middleware/
|   |   |-- auth.js                 # JWT verification & role inspection
|   |   \-- validate.js             # Defensive request payload validation
|   |-- models/
|   |   |-- User.js                 # Mongoose schema for system accounts
|   |   \-- DailyReading.js         # Schema for operational telemetry
|   |-- routes/
|   |   |-- auth.js                 # Authentication REST endpoints
|   |   |-- readings.js             # Daily reading CRUD REST routes
|   |   |-- dashboard.js            # KPI aggregation endpoints
|   |   |-- analytics.js            # Production analysis endpoints
|   |   \-- sustainability.js      # ESG compliance score endpoints
|   |-- .env                        # Environment secrets configuration
|   |-- package.json                # Backend dependencies specification
|   \-- server.js                   # Express server entry point
\-- frontend/
    |-- src/
    |   |-- components/
    |   |   |-- TopBar.jsx          # Header with branding and mobile drawer
    |   |   |-- Sidebar.jsx         # Navigation drawer with 44px tap targets
    |   |   |-- KPICard.jsx         # Animated metric card with sparkline bars
    |   |   |-- ChartContainer.jsx  # Recharts wrapper with scroll controls
    |   |   \-- AddReadingModal.jsx # Telemetry entry modal dialog
    |   |-- context/
    |   |   |-- AuthContext.jsx     # Client authentication state provider
    |   |   \-- ThemeContext.jsx    # Dark and light mode state provider
    |   |-- pages/
    |   |   |-- DashboardPage.jsx   # Main overview operations screen
    |   |   |-- AnalyticsPage.jsx   # Detailed efficiency & cost analytics
    |   |   |-- SustainabilityPage.jsx # ESG metrics and carbon offsets
    |   |   \-- AdminPage.jsx       # User management console
    |   |-- services/
    |   |   \-- api.js              # Axios HTTP client with JWT interceptors
    |   |-- App.jsx                 # Main React router and layout container
    |   \-- main.jsx                # React DOM mounting entry point
    |-- tailwind.config.js          # Responsive breakpoints and color tokens
    |-- vite.config.js              # Vite build tool configuration
    \-- package.json               # Frontend client dependencies
```

2. Environment Configuration (.env)

```
PORT=5000
NODE_ENV=production
MONGO_URI=mongodb+srv://admin:<password>@cluster0.mongodb.net/hydrotrack?retryWrites=true&
w=majority
JWT_SECRET=super_secret_jwt_cryptographic_key_hydrotrack_2026
JWT_EXPIRE=24h
CLIENT_URL=https://hydrotrack-client.vercel.app
```

Listing A.1: Sample Backend Environment Configuration

3. Database Connection Pool (backend/config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      maxPoolSize: 10,
      serverSelectionTimeoutMS: 5000,
      socketTimeoutMS: 45000,
    });
    console.log(`MongoDB Atlas Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Listing A.2: MongoDB Atlas Connection Module

4. User Mongoose Schema (backend/models/User.js)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Please provide full name'], trim: true },
  email: {
    type: String,
    required: [true, 'Please provide email address'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
  },
  password: { type: String, required: [true, 'Please provide password'], minlength: 6,
select: false },
  role: { type: String, enum: ['operator', 'admin'], default: 'operator' },
  createdAt: { type: Date, default: Date.now },
});

UserSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

module.exports = mongoose.model('User', UserSchema);
```

Listing A.3: User Mongoose Schema with Role Validation

5. DailyReading Mongoose Schema (backend/models/DailyReading.js)

```
const mongoose = require('mongoose');

const DailyReadingSchema = new mongoose.Schema({
  date: { type: String, required: [true, 'Date string is required (YYYY-MM-DD)'],
unique: true, index: true },
  hydrogenProduced: { type: Number, required: true },
  hydrogenConsumed: { type: Number, required: true },
  targetProduction: { type: Number, required: true, default: 1350 },
  energyConsumed: { type: Number, required: true },
  renewableEnergy: { type: Number, required: true },
```

```
  waterConsumed: { type: Number, required: true },
  electricityCost: { type: Number, required: true },
  waterCost: { type: Number, required: true },
  maintenanceCost: { type: Number, required: true },
  co2Emissions: { type: Number, required: true },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model('DailyReading', DailyReadingSchema);
```

Listing A.4: DailyReading Mongoose Schema with Indexed Date

6. Authentication Middleware (backend/middleware/auth.js)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401).json({ success: false, message: 'Not authorized to access
route' });
  }
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    if (!req.user) {
      return res.status(401).json({ success: false, message: 'User token no longer
valid' });
    }
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Token verification failed'
});
  }
};

exports.authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `User role '${req.user.role}' is not authorized to execute this
operation`,
      });
    }
    next();
  };
};
```

Listing A.5: JWT Verification and RBAC Middleware

7. Mathematical Calculations Engine (readingController.js)

```
const calculateReadingMetrics = (reading) => {
  const produced = reading.hydrogenProduced || 1;
  const target = reading.targetProduction || 1350;
  const totalEnergy = reading.energyConsumed || 1;
  const renewableEnergy = reading.renewableEnergy || 0;
  const totalCost = (reading.electricityCost || 0) + (reading.waterCost || 0) +
(reading.maintenanceCost || 0);

  // Formula 1: Production Efficiency
```

```
  const efficiency = Number(((produced / target) * 100).toFixed(1));
  // Formula 2: Cost per Kilogram
  const costPerKg = Number((totalCost / produced).toFixed(2));
  // Formula 3: Renewable Energy Share Percentage
  const renewableEnergyPct = Number(((renewableEnergy / totalEnergy) * 100).toFixed(1));
  // Formula 4: Specific CO2 Emissions per Kilogram
  const co2PerKg = Number(((reading.co2Emissions || 0) / produced).toFixed(3));

  return {
    ...reading.toObject(),
    operationalCost: totalCost,
    efficiency,
    costPerKg,
    renewableEnergyPct,
    co2PerKg,
  };
};
```

Listing A.6: Core Server-Side Calculation Logic

8. Sample REST API Payloads

```
{
  "date": "2026-10-17",
  "hydrogenProduced": 1380.5,
  "hydrogenConsumed": 1310.0,
  "targetProduction": 1350.0,
  "energyConsumed": 71.4,
  "renewableEnergy": 58.6,
  "waterConsumed": 14500,
  "electricityCost": 132000,
  "waterCost": 8100,
  "maintenanceCost": 18500,
  "co2Emissions": 590
}
```

Listing A.7: Sample JSON Ingestion Payload

```
{
  "success": true,
  "kpis": {
    "hydrogenProduced": { "value": 1380.5, "unit": "kg/day" },
    "hydrogenConsumed": { "value": 1310.0, "unit": "kg/day" },
    "productionEfficiency": { "value": 102.3, "unit": "%" },
    "operationalCost": { "value": 158600, "unit": "INR" },
    "costPerKg": { "value": 114.89, "unit": "INR/kg" },
    "renewableEnergyPct": { "value": 82.1, "unit": "%" }
  },
  "alerts": [
    {
      "id": "alt-01",
      "type": "optimal",
      "message": "Production efficiency exceeds plant benchmark target (102.3% load)."
    }
  ]
}
```

Listing A.8: Sample Evaluated Dashboard Response Payload

9. Client Authentication Context Provider (AuthContext.jsx)

```
import React, { createContext, useContext, useState, useEffect } from 'react';
import axios from 'axios';
```

```
const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(() => localStorage.getItem('hydrotrack_token') ||
null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (token) {
      axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;
      localStorage.setItem('hydrotrack_token', token);
    } else {
      delete axios.defaults.headers.common['Authorization'];
      localStorage.removeItem('hydrotrack_token');
    }
    setLoading(false);
  }, [token]);

  const login = async (email, password) => {
    const res = await axios.post('/api/auth/login', { email, password });
    setToken(res.data.token);
    setUser(res.data.user);
    return res.data;
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.removeItem('hydrotrack_token');
  };

  return (
    <AuthContext.Provider value={{ user, token, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

Listing A.9: Client-Side Authentication State Provider (AuthContext.jsx)

10. Defensive Request Validation Middleware (validate.js)

```
// backend/middleware/validate.js
exports.validateDailyReading = (req, res, next) => {
  const { date, hydrogenProduced, targetProduction, energyConsumed, renewableEnergy,
waterConsumed } = req.body;
  const errors = [];

  // ISO Date assertion (YYYY-MM-DD)
  if (!date || !/^\d{4}-\d{2}-\d{2}$/.test(date)) {
    errors.push('Date is required in valid ISO format (YYYY-MM-DD).');
  }

  // Non-negative physical boundary constraints
  if (typeof hydrogenProduced !== 'number' || hydrogenProduced < 0) {
    errors.push('Hydrogen produced must be a non-negative floating-point number (kg).');
  }
  if (typeof targetProduction !== 'number' || targetProduction <= 0) {
    errors.push('Target production must be a positive number greater than zero.');
  }
  if (typeof energyConsumed !== 'number' || energyConsumed < 0) {
    errors.push('Energy consumed must be non-negative (MWh).');
  }
  if (typeof renewableEnergy !== 'number' || renewableEnergy < 0 || renewableEnergy >
(energyConsumed || Infinity)) {
    errors.push('Renewable energy cannot be negative or exceed total energy consumed.');
  }
```

```
  if (typeof waterConsumed !== 'number' || waterConsumed < 0) {
    errors.push('Water consumed must be non-negative (Liters).');
  }

  if (errors.length > 0) {
    return res.status(400).json({ success: false, errors });
  }
  next();
};
```

Listing A.10: Defensive Request Validation Middleware (validate.js)

SCREENSHOTS OF THE PROJECT

This section presents the primary user interfaces of the HydroTrack platform, illustrating key workflows including user authentication, real-time KPI visualization, production analytics, environmental ESG monitoring, daily reading management, and administrative user governance.

Authentication Interfaces

The application provides streamlined, validated Login and Signup screens. The registration interface enforces password confirmation, email validation, and role initialization, as shown in Figure S.1.

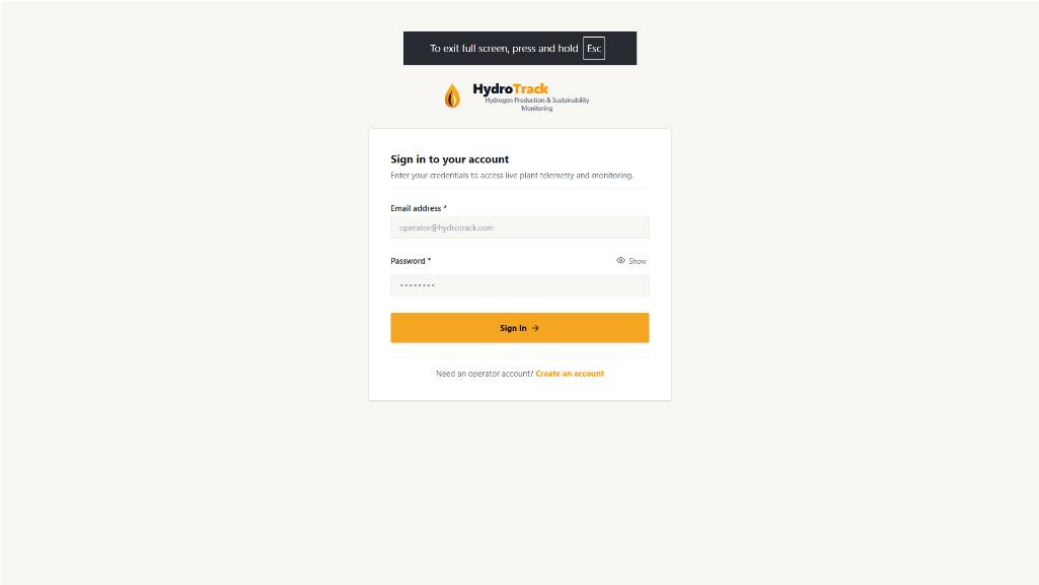


Figure S.1: Login and User Registration Interface

Operations Dashboard

The primary Dashboard displays high-level plant operational indicators, including daily hydrogen produced, consumption volume, production efficiency, and operating expenditure (Figure S.2). Dynamic charts display 7-day and 30-day production trends alongside target capacity thresholds.

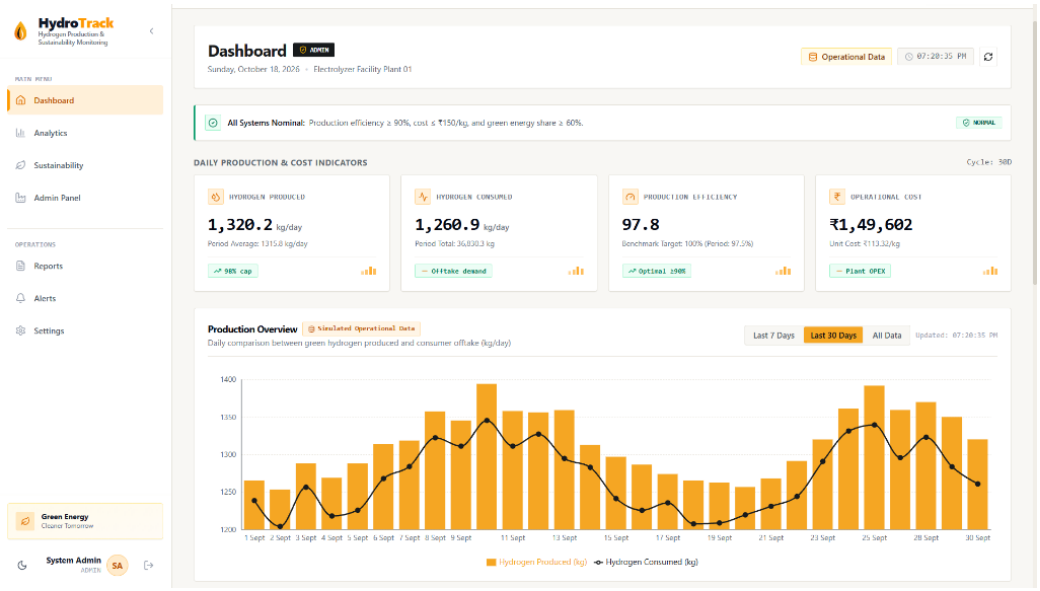


Figure S.2: HydroTrack Operations Dashboard with KPIs and Telemetry Charts

Analytics Module

The Analytics interface (Figure S.3) provides engineering telemetry breakdown, including load factors, target comparisons, specific energy consumption indices (MWh/kg), and itemized expenditure distributions across electricity, water demineralization, and stack maintenance.

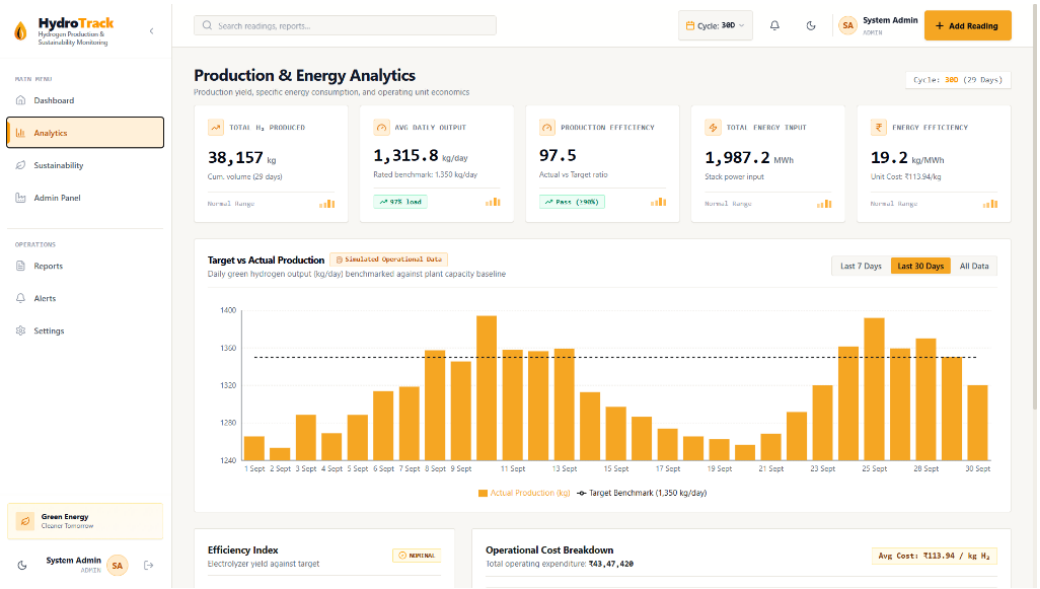


Figure S.3: Production Analytics and OPEX Distribution Interface

Sustainability and ESG Module

The Sustainability module (Figure S.4) computes and displays environmental benchmarks, including avoided greenhouse gas emissions, renewable energy percentages, water consumption efficiency, and cumulative ESG performance scores.

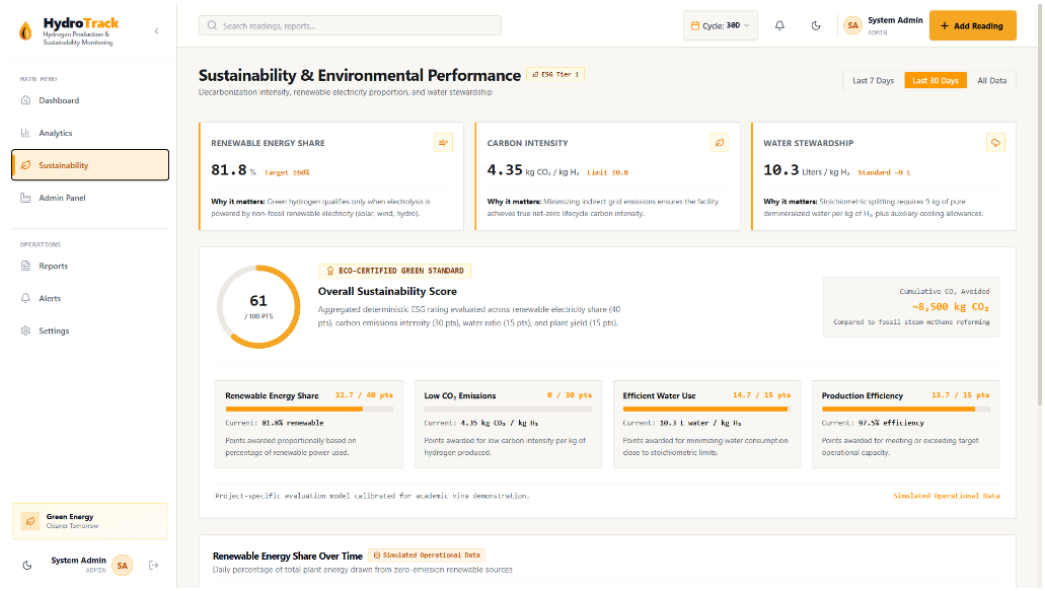


Figure S.4: Sustainability and Environmental Performance Interface

Daily Reading CRUD Modals

Operators and Administrators manage records through defensive modal dialogues. The Add/Edit Reading modal (Figure S.5) validates 11 engineering parameters with boundary checks before initiating server persistence.

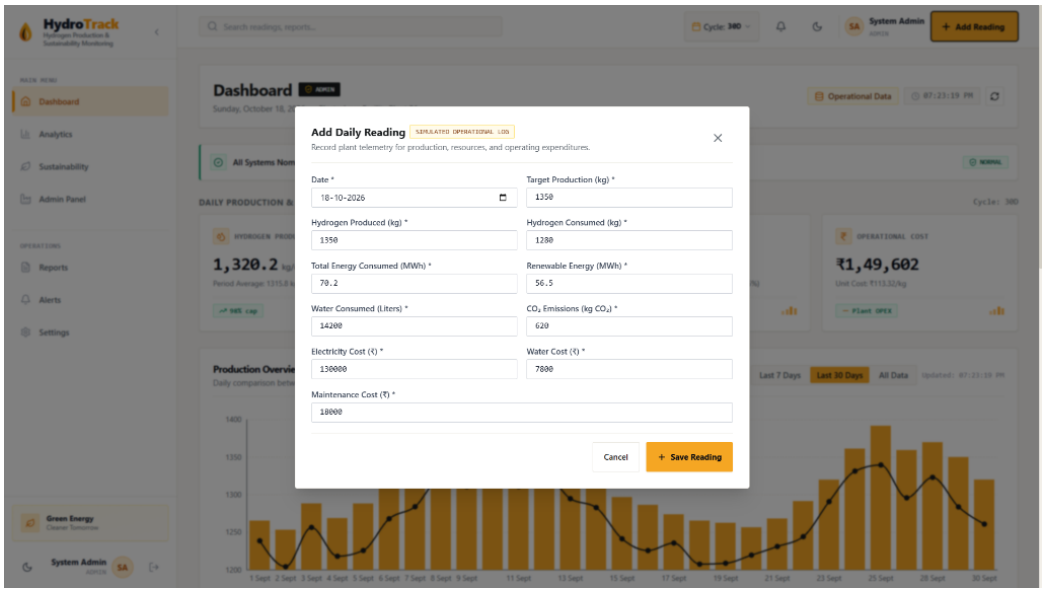


Figure S.5: Daily Telemetry Reading Management Modal

Administrative User Governance Panel

Administrators access a dedicated administrative console (Figure S.6) allowing inspection of all registered accounts, immediate toggling of roles between Operator and Administrator, and operator account termination with self-deletion protection.

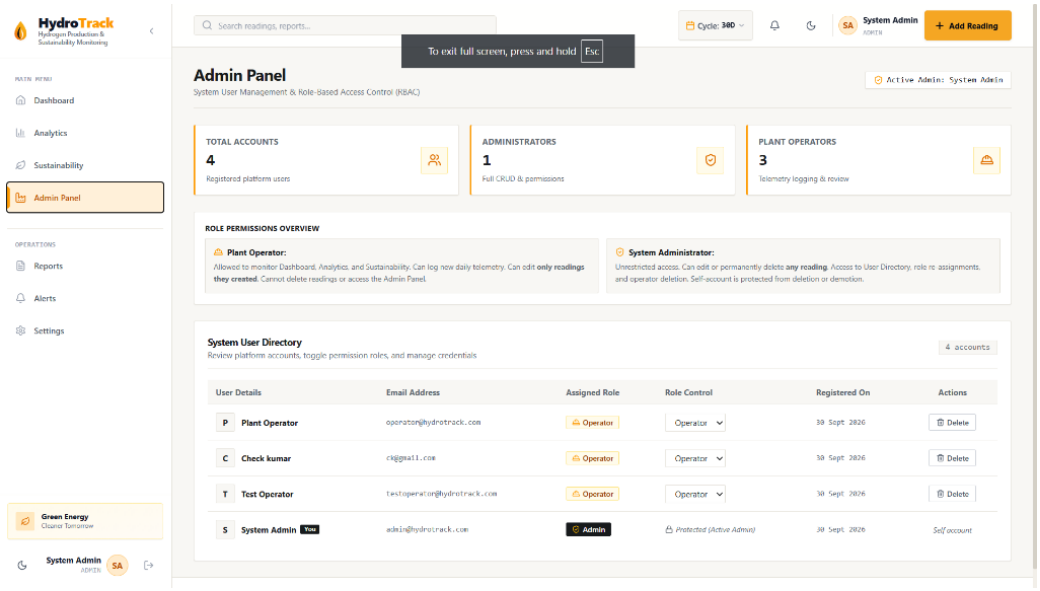


Figure S.6: System Administration and User Governance Panel

REFERENCES

[1] International Renewable Energy Agency, “Green hydrogen cost reduction: Scaling up electrolysers to meet the 1.5 °C climate goal,” IRENA, Abu Dhabi, UAE, Tech. Rep., 2020.

[2] U.S. Department of Energy, “Hydrogen and fuel cell technologies office multi-year program plan,” DOE, Washington, D.C., USA, Tech. Rep., 2021.

[3] O. Schmidt, A. Gambhir, I. Staffell, A. Hawkes, J. Nelson, and S. Few, “Future cost and performance of water electrolysis: An expert elicitation study,” International Journal of Hydrogen Energy, vol. 42, no. 52, pp. 30 470–30 492, 2017.

[4] International Energy Agency, Global Hydrogen Review 2023. Paris, France: IEA Publications, 2023.

[5] S. S. Kumar and V. Himabindu, “Hydrogen production by PEM water electrolysis – a review,” Materials Science for Energy Technologies, vol. 2, no. 3, pp. 442–454, 2019.

[6] E. Bank, “The MERN stack: MongoDB, Express, React, and Node.js for full-stack web development,” ACM Inroads, vol. 11, no. 4, pp. 45–52, 2020.

[7] K. Chodorow, MongoDB: The Definitive Guide: Powerful and Scalable Data Storage, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[8] A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Applications, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[9] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, 2010.

[10] M. Beller and A. Zaidman, “Continuous integration and continuous deployment in cloud environments,” IEEE Software, vol. 34, no. 5, pp. 24–28, 2017.